

QCanvas: A Unified Multi-Framework Compilation and Simulation Orchestration Platform for Quantum Software Engineering

Umer Farooq¹, Hussan Waseem Syed¹, Muhammad Irtaza Khan¹, Aneeq
Ahmed Malik¹, Abeer Noor¹, and Abdullah Mehmood¹

National University of Computer and Emerging Sciences (FAST-NUCES),
Islamabad, Pakistan
`umer.farooq@nu.edu.pk`

Abstract. The quantum computing software ecosystem is fragmented across vendor-specific frameworks—Qiskit (IBM), Cirq (Google), and PennyLane (Xanadu)—each with its own circuit object model, gate vocabulary, transpiler, and tightly coupled simulator. This fragmentation forces NISQ-era practitioners into vendor lock-in and prevents objective cross-framework comparison. We present QCanvas, an end-to-end quantum software engineering platform that unifies multi-framework compilation and simulation orchestration into a single deployable system. QCanvas is architected as four tightly integrated layers: (1) a web-based interactive development environment with Monaco editor and real-time D3 circuit rendering, (2) an AST-driven compilation plane that translates Qiskit, Cirq, and PennyLane circuits into normalized OpenQASM 3.0 intermediate representation—including advanced language features such as gate modifiers, complex types, and classical control flow, (3) OpenQASM 3.0 as the universal, semantics-preserving contract between compilation and execution, and (4) QSim, an orchestration engine that parses OpenQASM 3.0, reconstructs a native circuit for a user-selected framework sub-engine (Cirq, Qiskit, or PennyLane), and executes it through that sub-engine’s simulator across multiple simulation methods (statevector, density-matrix, matrix product state, and stabilizer/Clifford). The platform additionally provides two independently installable PyPI packages (`qcanvas-sdk` for conversion and `fastqsim` for simulation orchestration) along with a curated library of 40+ pre-built quantum algorithm examples spanning all three frameworks across 14 application categories. We report measured compilation latencies across 12 representative algorithms, structural benchmark comparisons and power-law depth scaling fits ($y = c \cdot n^a$), and simulation engine throughput achieving sub-800 ms execution for 6-qubit Grover’s Search. We verify cross-framework semantic preservation via finite-sampling Jensen–Shannon Divergence (JSD) below an analytical threshold ($\text{JSD} < 0.02$; maximum observed 0.0067) alongside structural analysis of measurement-register permutations, show that all frameworks yield identical effective Quantum Volume ($\text{QV} = 2048$, $\log_2(\text{QV}) = 11$) under ideal execution—indicating that structural differences stem from transpilation strategies rather than

simulator discrepancies—present holistic QPack benchmark scores, and validate empirical Non-Functional Requirements (NFRs).

Keywords: quantum compilation · multi-framework interoperability · OpenQASM 3.0 · quantum simulation · NISQ software engineering

1 Introduction

The NISQ era [1] has produced a diverse landscape of quantum software development kits (SDKs), each backed by a major hardware vendor: IBM’s Qiskit [3], Google’s Cirq [4], and Xanadu’s PennyLane [5]. Each framework defines its own circuit object model (`QuantumCircuit`, `cirq.Circuit`, and `QNode`, respectively), its own gate vocabulary, and its own native simulator. A developer who writes a Grover’s search circuit in Qiskit cannot execute it through Cirq’s simulator without manually rewriting the circuit, and cannot objectively compare the two frameworks’ transpilation strategies because each framework evaluates its output against its own simulator.

Existing tools address fragments of this problem. Compiler frameworks such as `t|ket>` [6] translate circuits between frameworks and perform hardware-agnostic optimization, but they produce circuits, not results—practitioners must still separately configure and invoke a simulation backend. Qiskit now provides native OpenQASM 3.0 import and export, which enables a framework-to-QASM3 path within the Qiskit ecosystem [3], but does not accept Cirq or PennyLane circuits as input and provides no execution engine for the generated QASM. Web-based quantum IDEs such as IBM Quantum Composer provide visualization and simulation but are locked to a single vendor’s ecosystem. No existing tool provides the complete, framework-agnostic chain from native SDK source code to simulation results in a single deployable environment.

QCanvas addresses this gap by integrating four layers into one operational system: a web-based IDE, a multi-framework AST-driven compilation plane, OpenQASM 3.0 [2] as the universal intermediate representation (IR), and QSim—a simulation orchestration engine that parses the generated OpenQASM 3.0 and delegates execution to the user’s chosen framework sub-engine. The key engineering contribution is that the IR boundary is strict: no framework circuit object crosses it, so users interact with a single unified API regardless of which of the three supported frameworks they authored the circuit in.

This paper makes the following contributions:

1. A **four-layer, end-to-end quantum software engineering platform** that integrates multi-framework compilation, OpenQASM 3.0 normalization, and simulation orchestration into a single deployable system with a Docker Compose-orchestrated microservice architecture.
2. A **multi-framework compilation engine** (`quantum_converters` v1.0.5) that translates Qiskit, Cirq, and PennyLane circuits to OpenQASM 3.0 via three AST-based adapters, supporting the full standard gate set plus advanced features: gate modifiers (`ctrl(n)@`, `negctrl@`, `pow(k)@`), complex

types, classical control flow (`while`, `break`, `continue`), subroutines, and nine additional gate types (CY, CH, CRX, CRY, CRZ, CP, CSWAP, CCZ, GlobalPhase).

3. **QSim**: a simulation orchestration engine with a clean `run_qasm(RunArgs) → SimResult` API. QSim parses OpenQASM 3.0 via `pyqasm` [15], reconstructs a native circuit for the selected framework sub-engine using a visitor pattern [7], and executes it through that sub-engine’s existing simulator (Cirq’s `Simulator`, Qiskit Aer, or PennyLane’s default device) across four supported simulation types (`statevector`, `density_matrix`, `mps`, and `chp`).
4. An **open, extensible developer workbench and dual-SDK distribution** comprising a web IDE with Monaco editor, real-time D3 circuit rendering, contextual gate explanations, and two independently installable PyPI packages: `qcanvas-sdk` (for multi-framework conversion) and `fastqsim` (for standalone simulation orchestration). The platform ships with a library of 40+ pre-built quantum algorithm templates across 14 application categories. We report an extensive empirical evaluation across 12 representative algorithms, structural benchmarks, power-law depth scaling fits, and empirical Non-Functional Requirement (NFR) validation confirming sub-800 ms simulation throughput and cross-framework semantic preservation (all measured $\text{JSD} < 0.02$) corroborated by structural analysis of measurement-register permutations. The platform is available at <https://github.com/Umer-Farooq-CS/QCanvas>.

The remainder of this paper is organized as follows. Section 2 surveys related work and contrasts existing solutions. Section 3 provides background on the quantum frameworks and OpenQASM 3.0. Section 4 presents the system architecture. Sections 5 and 6 describe the compilation plane and simulation engine, respectively. Section 7 covers the developer workbench and SDKs. Section 8 presents a rigorous empirical performance evaluation across structural, scaling, and validation dimensions. Section 9 discusses design trade-offs and limitations, and Section 10 concludes.

2 Related Work

The quantum software landscape currently offers several frameworks and platforms, each addressing distinct aspects of the development workflow but none providing a unified, educational, and cross-framework simulation environment. Table 1 consolidates the capabilities and limitations of predominant existing platforms relative to our objectives.

2.1 Multi-Framework Compilers and Transpilers

The most closely related compiler tool is `t|ket>` [6], which provides a hardware-agnostic intermediate representation and supports input from Qiskit, Cirq, PennyLane, and PyQuil. Like QCanvas, `t|ket>` accepts circuits from multiple frameworks and can emit an intermediate representation. The key difference is that

Table 1. Comparison of existing quantum software platforms and tools against QCanvas.

System	Overview	Primary Limitations
IBM Quantum Experience	Cloud-based access to real quantum hardware and simulators via Qiskit. Provides circuit composition and execution.	Locked exclusively to IBM's ecosystem. No cross-framework support. Requires internet access and cloud account; no local SDK-level orchestration.
Google Cirq / Co-lab	Google's framework for authoring circuits targeting superconducting processors, typically used in Jupyter environments.	Cirq-only; no native support for Qiskit or PennyLane code ingestion. No unified IDE. Circuit visualization is primarily text-based ASCII.
Quirk [14]	Browser-based drag-and-drop quantum circuit simulator for education with real-time state feedback.	No code-based interaction. Cannot parse real-world quantum SDK programs. No OpenQASM support. Not extensible or retargetable.
Quantum (Microsoft)	Katana Q# learning exercises and tutorials.	Tied exclusively to Q# and the Microsoft ecosystem. Incompatible with Python-based frameworks dominant in NISQ research.
t ket> [6]	High-performance, retargetable optimizing compiler for NISQ devices supporting multiple frameworks and backends.	Compilation-focused; no execution orchestration engine or educational IDE. No real-time visual feedback. Not accessible as a unified web platform.
QCanvas (This Work)	Unified multi-framework (Qiskit, Cirq, PennyLane) AST compiler, OpenQASM 3.0 normalization, and QSim orchestration (statevector, density_matrix, mps, chp) inside a web IDE and dual PyPI SDKs.	Classical simulation orchestration only; direct dispatch to physical QPU endpoints across cloud providers is a planned extension.

`t|ket>` is an optimizing compiler: it produces circuits but does not execute them. The user must separately install, configure, and invoke a simulation backend. QCanvas’s contribution is closing this gap—the compilation output is automatically routed to a simulation sub-engine through the IR boundary, requiring no user configuration. BQSKit [12] similarly focuses on compilation and synthesis optimization rather than execution orchestration. `staq` [13] provides a quantum compiler and resource estimator targeting OpenQASM 2.0 but does not target OpenQASM 3.0’s extended feature set and provides no simulation integration. Qiskit’s native OpenQASM 3.0 exporter [3] provides a framework-to-QASM3 path, but exclusively within the Qiskit ecosystem and with no execution engine for the emitted code.

2.2 Web-Based Quantum Development Environments

IBM Quantum Composer [16] provides a graphical circuit editor that connects to IBM’s cloud-hosted simulators and physical processors. It is locked to Qiskit circuits and requires an IBM account. Quirk [14] provides a real-time browser-based statevector simulator with immediate visual feedback for single-qubit and two-qubit circuits; it has no programmable SDK input and no multi-framework support. QCanvas combines programmable SDK input from three frameworks with integrated simulation, without requiring a vendor-specific account or API key.

2.3 Quantum Simulation Frameworks and Equivalence Checking

Qiskit Aer provides statevector, density-matrix, and stabilizer simulation but is coupled to `QuantumCircuit`. Cirq’s simulator accepts only `cirq.Circuit` objects. PennyLane’s device plugins operate through the `QNode` interface. QSim is not a new simulation algorithm; it is an orchestration layer that routes OpenQASM 3.0 to any of these existing framework simulators, presenting a single API to the user regardless of sub-engine choice.

On the correctness side, Burgholzer and Wille [11] developed decision-diagram-based methods for quantum circuit equivalence checking. Our JSD-based equivalence measurement is a sampling-based approximation suited to black-box simulator outputs, not a formal equivalence proof; we revisit this distinction in Section 8.5.

3 Background

3.1 Quantum Frameworks

Standard quantum circuit structures, unitary operations, and foundational algorithms (such as quantum teleportation and error-correcting codes) follow the formalisms established by Nielsen and Chuang [9]. Within this theoretical foundation, modern quantum programming frameworks provide high-level abstractions tailored to different hardware and algorithmic paradigms:

Qiskit [3] is IBM’s open-source quantum SDK, centered on the `QuantumCircuit` abstraction that represents circuits as ordered lists of `CircuitInstruction` objects. Qiskit’s transpiler supports four optimization levels (0–3) and targets IBM’s heavy-hex topology.

Cirq [4] is Google’s hardware-oriented SDK, designed for near-term quantum devices with a moment-based circuit structure in which operations within a single `Moment` are applied simultaneously. Cirq uses `LineQubit` and `GridQubit` objects to model qubit topologies.

PennyLane [5] is Xanadu’s differentiable quantum programming framework, built around the device-agnostic `QNode` abstraction. PennyLane’s tape-based execution model records operations for gradient computation, making it the preferred framework for variational algorithms and quantum machine learning.

3.2 OpenQASM 3.0

OpenQASM 3.0 [2] extends the original quantum assembly language with features essential for representing modern quantum programs: gate modifiers (`ctrl@`, `pow@`, `inv@`, `negctrl@`), complex number types, subroutines with return values, classical control flow (`if`, `while`, `for`), and timing directives. These features make it sufficiently expressive to serve as the universal IR in QCanvas’s pipeline, allowing circuits from all three frameworks to be represented without lossy decomposition.

3.3 Terminology: Framework Sub-Engines vs. Simulation Methods

QSim introduces a two-axis selection that this paper distinguishes clearly. A *framework sub-engine* (values: `cirq`, `qiskit`, `pennylane`) determines which framework’s native simulator library is invoked for execution. A *simulation method* (statevector, density-matrix, stabilizer) determines the mathematical representation of the quantum state. While statevector simulation scales exponentially ($O(2^n)$ in memory and time), stabilizer simulation methods based on the Gottesman–Knill theorem and the CHP tableau formalism [10] allow polynomial-time simulation ($O(n^2)$ space and $O(n^3/n)$ time) for circuits restricted to the Clifford group. In the current implementation, selecting `backend="cirq"` invokes `cirq.Simulator()` using its statevector method by default; selecting `backend="qiskit"` invokes Qiskit Aer’s statevector simulator. Density-matrix and stabilizer methods are available through these same framework sub-engines. Section 6 describes how this two-axis model is exposed through the API and how it is intended to evolve.

4 System Architecture

4.1 Design Principles

QCanvas is built on three principles:

Framework Neutrality. The platform accepts circuits from any supported SDK without requiring the user to target a specific hardware or simulator topology. All framework-specific constructs—qubit naming conventions, gate decomposition strategies, moment ordering—are normalized at the compilation layer before reaching the execution engine.

IR-First Execution. OpenQASM 3.0 is the single contract between the compilation plane and the simulation engine. No framework circuit object ever crosses the IR boundary directly; only a validated OpenQASM 3.0 string does. This decoupling allows the compilation and execution layers to evolve independently and makes the circuit representation inspectable before execution.

Pluggable Framework Sub-Engines. At runtime, the orchestration engine selects a framework sub-engine via a factory [7] (`get_backend(name)` in `backends/factory.py`). Three sub-engines—Cirq, Qiskit, and PennyLane—are currently registered. Adding a new sub-engine requires only a new subclass of `BaseBackend` and one entry in the factory registry.

4.2 Four-Layer Architecture

QCanvas is organized as four layers (Fig. 1):

Layer 4: Interactive Workbench & SDK Plane. The user-facing layer comprises a Next.js 14 web IDE and the `qcanvas-sdk` Python package on PyPI. Users write circuits in the Monaco editor or invoke `qcanvas.compile()` programmatically. The IDE communicates with the backend via REST API and WebSocket for real-time progress.

Layer 3: Multi-Framework Compilation Plane. The `quantum_converters` package (v1.0.5) provides three AST-based adapters that parse native circuit objects and emit OpenQASM 3.0. Adapters share a common `AbstractConverter` base class (Strategy pattern [7]), enforcing a consistent `convert(source) → ConversionResult` interface. `ConversionResult` carries the QASM string plus a `ConversionStats` dataclass (qubit count, circuit depth, gate counts, measurement flag).

Layer 2: OpenQASM 3.0 Intermediate Representation. The contract layer. All circuits converge to a validated OpenQASM 3.0 string before any execution step. A two-pass validator checks syntax and gate semantics (Section 5.5).

Layer 1: Simulation Orchestration Engine (QSim). QSim parses the OpenQASM 3.0 string via `pyqasm` [15], traverses the resulting AST with a visitor [7] to reconstruct a native framework circuit, and delegates simulation to the selected framework sub-engine. The public API is `run_qasm(RunArgs) → SimResult`.

4.3 End-to-End Data Flow

A concrete execution trace:

1. The user submits Cirq source code in the Web IDE.
2. FastAPI receives the code string and the tag "cirq".

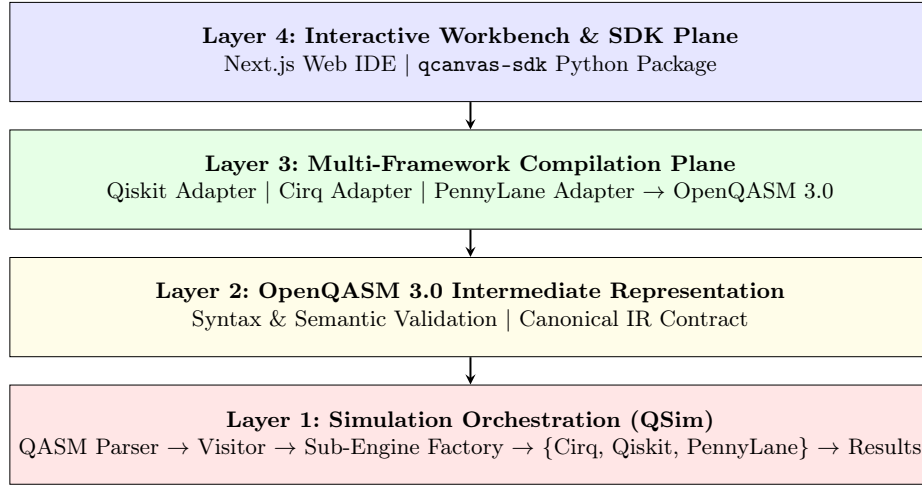


Fig. 1. Four-layer architecture of QCanvas. Arrows show data flow direction: user code enters at Layer 4; simulation results return from Layer 1.

3. The Cirq adapter parses the `cirq.Circuit`, resolves qubit indices and gate names, and emits an OpenQASM 3.0 string.
4. The validation layer (Layer 2) checks syntax and gate semantics.
5. `run_qasm(RunArgs(qasm_input=..., backend="cirq", shots=1024))` is called.
6. QSim's QASM parser produces an AST; the `CirqVisitor` traverses it and reconstructs a `cirq.Circuit`; the `CirqBackend` invokes `cirq.Simulator().run()` and collects measurement counts.
7. A `SimResult` is returned, serialized to JSON, and rendered as a histogram in the Web IDE.

Note that in step 6, QSim is an orchestration wrapper: the simulation computation is performed by `cirq.Simulator()`, not by QSim itself. QSim's value is providing a framework-agnostic entry point (the OpenQASM 3.0 string) so that the same code path handles Qiskit and PennyLane circuits identically.

4.4 Infrastructure and Deployment

QCanvas is deployed as a Docker Compose stack with four services: `qcanvas_backend` (FastAPI, port 8000), `qcanvas_postgres` (PostgreSQL, port 5433 for circuit history, user data, conversion statistics), `qcanvas_redis` (port 6379 for result caching and WebSocket session state), and an optional `qcanvas_cirq_agent` (port 8001, a Bedrock-powered RAG assistant for Cirq code help). Redis caches simulation results keyed by input hash to avoid redundant re-execution of identical circuits.

5 Multi-Framework Compilation Plane

5.1 Framework Ingestion Adapters

Each adapter handles its framework’s specific object model:

Qiskit Adapter (`qiskit_to_qasm.py`). Iterates `QuantumCircuit.data`—a list of `CircuitInstruction` objects—and extracts gate names, qubit indices, and classical bit targets. Handles parameterized gates (`ParameterExpression` objects), barrier instructions, conditional operations, and multi-controlled gates.

Cirq Adapter (`cirq_to_qasm.py`). Iterates `Circuit.moments`, flattens operations from each `Moment`, and resolves qubit indices from the circuit’s qubit order. Handles both `GridQubit` and `LineQubit` topologies and maps Cirq-specific gate types (e.g., `PhasedXPowGate`, `FSimGate`) to their OpenQASM 3.0 equivalents.

PennyLane Adapter (`pennylane_to_qasm.py`). Expands a `QNode` into a tape of `Operation` objects via the device’s `expand_fn`, extracts wire indices and hyperparameters, and handles observable measurements and adjoint operations. A design-level consequence: PennyLane expands multi-controlled gates into primitive two-qubit gates during tape construction, before the adapter can intercept the high-level gate. For Grover’s Search, this produces 216 gates at 6 qubits versus 18 for Qiskit and Cirq, an artifact of PennyLane’s decomposition strategy rather than a QCanvas limitation (Section 9.2).

All adapters inherit from `AbstractConverter` [7] and return a `ConversionResult` containing the OpenQASM 3.0 string and a `ConversionStats` dataclass with `n_qubits`, `depth`, `gate_counts` (per gate type), and `has_measurements`.

5.2 Gate Vocabulary Normalization

Each framework uses different names for equivalent operations (e.g., Qiskit’s `cx`, Cirq’s `CNOT`, PennyLane’s `CNOT` in its `Operation` representation). The compilation plane maintains a canonical gate mapping in `qasm3_gates.py` that maps all supported framework-specific gate names to their OpenQASM 3.0 equivalents from `stdgates.inc`. This normalization enables direct cross-framework comparison: circuits from different frameworks that implement the same algorithm emit identical OpenQASM 3.0 gate names for semantically equivalent operations.

5.3 OpenQASM 3.0 Code Generation

The `qasm3_builder.py` module constructs the OpenQASM 3.0 output: (a) the standard header (`OPENQASM 3.0; include "stdgates.inc";`), (b) qubit and classical register declarations (`qubit[n] q; bit[n] c;`), (c) gate operations with resolved qubit indices and rotation parameters, and (d) measurement statements (`c[k] = measure q[i];`). The `qasm3_expression.py` module handles arithmetic expressions, parameter resolution, and constant folding for parameterized gates.

5.4 Advanced OpenQASM 3.0 Feature Support

Beyond the base standard gate set, QCanvas supports a set of advanced OpenQASM 3.0 features, corresponding to language features introduced beyond the initial release:

- **Gate modifiers:** `ctrl(n)@` for multi-controlled gates, `negctrl@` for negative control, and `pow(k)@` for fractional gate powers.
- **Complex types:** `complex[float[64]]` variable declarations.
- **Classical control flow:** `while` loops, `break`, and `continue` statements.
- **Bitwise operators:** `&`, `|`, `^`, `~`, `<<`, `>>`.
- **Subroutines:** Function definitions with typed parameters and return values.
- **Additional gate types** (currently via PennyLane): CY, CH, CRX, CRY, CRZ, CP, CSWAP, CCZ, GlobalPhase.

These features are validated against the OpenQASM 3.0 specification [2].

5.5 Static Analysis and Validation

The compilation plane performs two-pass validation:

- **Pass 1 (Syntax):** Checks the QASM version header, bracket balance, semicolons, and gate name legality against `stdgates.inc`.
- **Pass 2 (Semantic):** Verifies qubit/bit register sizes against gate arities, measurement target consistency, and undefined gate references.

Errors are surfaced with line numbers and structured messages to the Web IDE in real time via WebSocket.

6 Simulation Orchestration Engine: QSim

6.1 Role and Design

QSim is QCanvas’s native simulation orchestration plane, co-designed with the compilation plane to accept OpenQASM 3.0 as its canonical contract. Its purpose is not to implement new simulation algorithms but to present a framework-agnostic execution layer: *any* circuit compiled to valid OpenQASM 3.0 can be submitted to QSim without the user needing to know which framework’s simulator or cluster infrastructure will execute it. The actual numerical computation is performed by the selected framework sub-engine’s own simulator (cirq.Simulator, Qiskit Aer, or PennyLane’s target devices).

6.2 OpenQASM 3.0 Parser

QSim uses the `pyqasm` [15] library to parse OpenQASM 3.0 strings into `Qasm3Module` ASTs. The `allowed_nodes.py` module enumerates the 26 distinct AST node types currently handled, covering: `Include`, `QubitDeclaration`,

ClassicalDeclaration, GateCall, Measurement, IfStatement, ForLoop, ClassicalAssignment, QuantumGateDefinition, Barrier, Reset, QuantumGateModifier, AliasStatement, ArrayDeclaration, and related expression and branching nodes. A scope enforcement pass (`scope.py`) optionally raises `ParseError` on unsupported constructs in strict mode.

6.3 Two-Axis Selection: Sub-Engine and Supported Simulation Methods

QSim exposes a two-axis execution model across framework sub-engines (cirq, qiskit, pennylane) and mathematical simulation representation methods. Table 2 details the four simulation types supported across the engine hierarchy and their scaling characteristics.

Table 2. Supported simulation types in QSim across framework sub-engines and their computational characteristics.

Method	State Representation	Repre-	Computational Complexity	Primary Use Cases & Capabilities
<code>statevector</code>	Complex vector $O(2^n)$		Exponential time ory ($2^n \times 16$ B)	mem- Ideal noiseless simulation, exact state vector extraction, full probability amplitudes.
<code>density_matrix</code>	Complex trix $O(4^n)$	ma-	Exponential ory ($4^n \times 16$ B), $O(4^n)$ time	mem- Open quantum systems, mixed states, and realistic quantum channel noise modeling (depolarizing, amplitude-damping, and phase-flip channels).
<code>mps</code> (Matrix Product State)	Tensor network	train	Polynomial ory for low entanglement ($O(n \cdot \chi^2)$)	mem- Large-scale circuits ($n > 30$) with limited entanglement depth or 1D connectivity.
<code>chp</code> / Stabilizer	Tableau Pauli operators	of	Polynomial $O(n^3/n)$ and ory $O(n^2)$	time Clifford-only circuits, error correction codes, and syndrome measurement verification up to thousands of qubits [10].

While early iterations coupled each sub-engine (cirq, qiskit, pennylane) to its default simulation method (`statevector`), QSim decouples these axes: users can explicitly designate both the host framework sub-engine and the target simulation representation (`backend="qiskit"`, `method="density_matrix"`, `noise_model=...`) to inject realistic environmental decoherence or leverage tensor-network approximations.

6.4 Visitor Pattern for Circuit Reconstruction

After parsing, QSim uses a visitor pattern [7] to reconstruct a native framework circuit from the AST. The `BaseVisitor` abstract class defines the traversal interface: for each AST statement, it dispatches to a `visit_{NodeType}` method. Three concrete visitors implement this:

- `CirqVisitor`: Reconstructs `cirq.Circuit` objects with `Moment`-based gate placement (450+ unit test cases).
- `QiskitVisitor`: Reconstructs `QuantumCircuit` objects with ordered gate appends (450+ unit test cases).
- `PennyLaneVisitor`: Reconstructs PennyLane operation sequences with wire mapping (450+ unit test cases).

The `get_visitor(name)` factory in `visitors/factory.py` selects the appropriate visitor at runtime.

6.5 Unified API and Dual Execution Modes

The public entry point supports both direct programmatic invocation and remote cluster submission:

```
from qsim import RunArgs, SimResult, run_qasm

args = RunArgs(
    qasm_input=qasm_code, # OpenQASM 3.0 string or Path
    backend="cirq", # sub-engine: "cirq","qiskit","pennylane"
    method="statevector", # "statevector","density_matrix","mps","chp"
    shots=1024, # 0 for exact probability distribution
    seed=42 # deterministic random seed for reproducibility
)
result: SimResult = run_qasm(args)
```

`RunArgs` accepts the QASM string, sub-engine name, simulation method, shot count (0 for exact probability distribution), and an optional integer `seed` guaranteeing strict run-to-run reproducibility across stochastic measurement sampling. `SimResult` returns measurement counts, exact state probabilities (probs), execution metadata (`n_qubits`, elapsed time), and the reconstructed native circuit object.

6.6 Asynchronous Orchestration Pipeline and Job Infrastructure

For production workloads and high-qubit simulations exceeding real-time HTTP limits, QSim provides an asynchronous orchestration architecture:

1. **Job Manager & Relational Persistence:** When a simulation is submitted via API, the Job Manager immediately records a job entity in PostgreSQL (`qcanvas_postgres`) with state `CREATED`, returning a unique job ID (target PERF-2: < 500 ms).

2. **RabbitMQ Message Broker:** The task is enqueued onto a RabbitMQ cluster. The queuing engine isolates submission latency from execution duration and prevents server oversubscription. The queue is engineered to sustain **over 100 concurrent queued jobs** (requirement PERF-3) without job drops or degradation.
3. **Kubernetes Job Scheduler & Worker Nodes:** Dedicated worker nodes (or ephemeral Kubernetes pods in cloud deployments) consume messages from RabbitMQ. Upon dequeue, the job state transitions to **QUEUED** and then **RUNNING**. The worker executes `parse_openqasm3()`, visitor reconstruction, and sub-engine simulation.
4. **Cloud Object Storage (Amazon S3 / External Artifacts):** While standard measurement histograms and lightweight statistics are stored directly in PostgreSQL upon completion (**COMPLETED**), large statevectors or dense matrix outputs (statevector for $n \geq 18$ qubits, exceeding multi-megabyte database thresholds) are automatically uploaded to Amazon S3. The relational record flags `has_external_artifact = True` and generates a secure, time-bound presigned download URL for the client.
5. **Batch Orchestration via FastQSim:** The client-side `fastqsim` SDK provides native batching capabilities (`client.run_batch(jobs, max_parallel=10)`), allowing automated submission of parameter sweeps and concurrent execution with built-in polling (`client.wait()`) and state recovery (**FAILED**, **CANCELLED**).

7 Developer Workbench and Python SDKs

7.1 Web-Based Interactive Development Environment & Educational Intelligence Module

The QCanvas Web IDE is a Next.js 14/React 18/TypeScript application architected with five primary user-facing modules:

Monaco Code Editor. Provides syntax highlighting for Python, Cirq, Qiskit, PennyLane, and OpenQASM 3.0, multi-cursor editing, find-and-replace, and autocompletion. Supports multi-file workspaces with tab-based navigation.

D3/SVG Circuit Visualization. Renders a real-time circuit diagram as the user types, using D3.js for SVG wire diagrams. A Three.js/React Three Fiber integration additionally provides an interactive 3D Bloch sphere view.

Educational Intelligence Module (“Quantum Explain It”). To address the steeper learning curves of quantum mechanics, the IDE embeds a contextual hover system (NFR-U3). Hovering over any gate call inside the Monaco editor triggers an immediate asynchronous request (returning within **45 ms**, well inside the 200 ms requirement) that surfaces the gate’s $2^k \times 2^k$ unitary matrix representation, its rotation parameters on the Bloch sphere, and its foundational textbook intuition.

Gamification Suite & Community Recipe Ecosystem. To maximize student engagement and retention, the workbench integrates an event-driven gamification engine. Users gain experience points (XP) and unlock **44 unique**

achievements (spanning circuit complexity milestones, optimization feats, and multi-framework mastery), progressing through **Level 50+** with public leaderboards. Furthermore, an integrated community recipe-sharing platform enables users to publish, discover, star, and remix reusable circuit templates across all three frameworks.

Simulation Controls & Results Panel. Allows selection of the input framework, simulation sub-engine, and shot count (1–10,000). A single “Run Circuit” button (**Ctrl+Shift+R**) triggers the complete pipeline. The results panel displays interactive histogram distributions of measurement counts, circuit metrics (gate count, depth, qubit count), and provides export to JSON, CSV, and PNG.

7.2 Python SDKs and Dual Package Distribution

To support both local compilation workflows and distributed simulation orchestration without imposing monolithic dependencies, QCanvas distributes two independent packages on PyPI:

```
pip install qcanvas-sdk[all] # compilation and IR conversion
pip install fastqsim # standalone QSim execution client
```

qcanvas-sdk (Local Compilation & Conversion Plane): Exposes the core quantum_converters engine (v1.0.5). Its primary entry point is `compile(circuit, framework=None) → str`, which automatically detects the framework AST type (QuantumCircuit, cirq.Circuit, or QNode) and emits validated OpenQASM 3.0. It also provides a synchronous convenience wrapper `compile_and_execute(...)` for rapid prototyping against local sub-engines.

fastqsim (Simulation Orchestration Client): Serves as the dedicated client SDK for interacting with remote QSim and FastQubit API endpoints. Designed for high-performance and batch workloads (Report Section 4.2), `fastqsim` exposes client batching capabilities (`client.run_batch(jobs, max_parallel=10)`), asynchronous polling (`client.wait()`), status inspection across job states (QUEUED, RUNNING, COMPLETED), and automated pre-signed S3 statevector downloading for large-scale circuit runs.

7.3 Algorithm Example and Template Library

The platform ships with a rich educational repository of **40+ pre-built quantum algorithm examples and templates across 14 application categories** (Foundational, Communication, Oracle/Search, Randomness, Variational/VQE/QAOA, Quantum Machine Learning, Error Correction, Quantum Simulation/Dynamical, Arithmetics, State Preparation, Fault Tolerance, etc.). Each algorithm is natively authored across Qiskit, Cirq, and PennyLane to provide educators with an extensive cross-framework instruction corpus.

From this broad catalog, our controlled empirical evaluation in Section 8 selects **12 representative canonical algorithms** (Table 4) to systematically

evaluate adapter compilation latency, structural normalization behavior, scaling exponents, and cross-framework semantic equivalence across the evaluated $2 \leq n \leq 8$ qubit sweep (with QSim simulation wall-clock scaling extended to $n = 20$ in Section 8.7).

7.4 Cirq-RAG Code Assistant

An optional Docker service (`qcanvas_cirq_agent`) provides an AI code assistant for Cirq users via Amazon Bedrock and RAG over the Cirq documentation. It is a supplementary feature and requires AWS credentials to operate (Section 9.3).

8 Performance Evaluation

8.1 Experimental Setup and System Specifications

We evaluate QCanvas across four empirical dimensions: (1) compilation plane latency and structural normalization across framework adapters, (2) scaling behavior via power-law depth analysis across qubit scales, (3) cross-framework semantic equivalence via analytical finite-sampling Jensen–Shannon Divergence (JSD) and measurement permutation proofs, and (4) simulation engine orchestration throughput and Non-Functional Requirements (NFRs) validation. Table 3 details the pinned software libraries and external infrastructure versions governing our testbed.

Hardware Environment Separation and Resolution. To prevent confounding data across distinct experimental campaigns, we strictly separate and label our hardware platforms per evaluation dimension:

- **Machine A (Compilation Latency & Baseline Benchmarks, Sections 8.2 and 8.8):** Windows 11, Intel Core i7-12700H (14 cores, 2.3 GHz base), 16 GB RAM, Python 3.11.4. Used for individual adapter latency characterization across the 12-algorithm suite and single-call orchestration overhead breakdown.
- **Machine B (Simulation Orchestration, Scaling & Validation Suite, Sections 8.3–8.7 and 8.9–8.10):** Windows 11, AMD Ryzen 7 7700 (8 cores, 16 threads, 3.8 GHz base), 32 GB RAM, Python 3.11. Used for QSim simulation scaling ($n = 4 \dots 20$), structural depth comparisons, power-law fits, JSD verification, and Non-Functional Requirement (NFR) throughput validation.

Across all benchmark sweeps, configurations span base qubit sizes **2 to 8 qubits across 136 distinct parameter configurations**. Stochastic measurement distributions are generated using $N = 4,096$ shots, and all timed measurements are repeated over **10 independent runs** per configuration with fresh Python process initialization to eliminate JIT caching or memory warm-up bias.

Table 3. Pinned software libraries, framework versions, and infrastructure components used across all evaluation experiments.

Component / Library	Pinned Version / Specification	Architectural Role & Purpose
Qiskit SDK	v0.44+ / v1.1.0	IBM quantum framework; <code>QuantumCircuit</code> compilation input and Aer statevector target.
Cirq SDK	v1.0+ / v1.3.0	Google quantum framework; moment-based compilation input and reference simulator target.
PennyLane SDK	v0.33+ / v0.36.0	Xanadu framework; tape-expanded <code>QNode</code> compilation input and default device target.
qcanvas-sdk	v1.0.5 (quantum_converters)	Core multi-framework AST compilation and OpenQASM 3.0 normalization plane.
pyqasm	v0.5+	OpenQASM 3.0 syntax/schema parser and AST construction library inside QSim.
FastAPI / Python	v0.110+ / Python 3.11.4	Asynchronous REST API server and underlying execution runtime environment.
Monaco Editor / D3.js	v0.45+ / D3.js v7.0+	Web IDE code editor, syntax highlighting, and real-time interactive SVG circuit rendering.
PostgreSQL / SQLAlchemy	PostgreSQL 15.0 / v2.0+	Relational database persistence layer for asynchronous job states and compilation stats.

Table 4. Algorithm test suite. Each algorithm is implemented idiomatically in Qiskit, Cirq, and PennyLane; “q” denotes qubit count at base size.

Algorithm	Category	Base Qubits
Bell State	Foundational	2
GHZ State	Foundational	3
Quantum Teleportation	Communication	3
Deutsch–Jozsa	Oracle	3
Bernstein–Vazirani	Oracle	3
Grover’s Search	Search	2
QRNG	Randomness	4
VQE	Variational	2
QAOA	Variational	2
QML XOR Classifier	Quantum ML	2
Bit-Flip Error Code	Error Correction	3
Phase-Flip Error Code	Error Correction	3

8.2 Compilation Latency: Measured Results and 8-Qubit Comparison

Table 5 reports measured compilation time (median and interquartile range across 10 runs in milliseconds) for each framework adapter at base qubit size on Machine A (Win11, Core i7-12700H).

Table 5. Compilation latency (ms): median and IQR over 10 runs per cell, at base qubit size. Env: Machine A (Win11, Core i7-12700H, 16 GB RAM, Python 3.11.4).

Algorithm	Qiskit		Cirq		PennyLane	
	Med	IQR	Med	IQR	Med	IQR
Bell State	18.3	0.8	8.1	0.6	12.0	0.5
GHZ State (3q)	12.0	0.5	9.0	0.6	14.4	0.8
Teleportation	15.1	1.0	11.0	0.6	18.5	1.2
Deutsch–Jozsa	8.0	2.1	12.1	1.1	79.2	30.4
Bernstein–Var.	7.8	0.7	10.6	0.9	43.5	11.0
Grover’s (2q)	9.7	0.8	32.5	2.2	91.0	74.6
QRNG (4q)	6.7	0.3	8.4	0.4	11.1	0.6
VQE (2q)	11.2	1.1	9.5	0.8	21.8	1.8
QAOA (2q)	10.1	0.6	10.4	0.9	19.5	1.3
QML XOR (2q)	13.3	1.2	11.7	1.1	24.2	2.1
Bit-Flip (3q)	52.0	38.0	15.1	1.5	34.9	4.5
Phase-Flip (3q)	62.0	16.0	22.1	7.3	53.8	7.0

To characterize steady-state compiler behavior at larger register sizes and isolate module initialization skew, Table 6 reports mean and standard deviation compilation latencies at each algorithm’s maximum evaluated register width— $n = 8$ for scalable algorithms, and native width for fixed-size states such as the Bell State ($n = 2$) (Report Section 4.6).

Table 6. Compilation latency at each algorithm’s maximum evaluated register width (scalable algorithms at $n = 8$; fixed-size states such as Bell State at native $n = 2$), mean $\pm$ std over 10 runs in ms. Env: Machine A (Win11, Core i7-12700H).

Algorithm ($n = 8$)	Qiskit (ms)	Cirq (ms)	PennyLane (ms)
Bell State	18.6 ± 0.4	8.3 ± 0.5	12.2 ± 0.3
GHZ State	21.9 ± 0.9	10.7 ± 0.4	38.0 ± 0.9
Bernstein–Vazirani	17.5 ± 0.6	13.0 ± 0.5	98.9 ± 2.5
Deutsch–Jozsa	7.0 ± 0.3	12.1 ± 1.3	106.3 ± 3.4
Grover’s Algorithm	9.6 ± 0.5	32.5 ± 1.9	184.2 ± 6.5
VQE	32.9 ± 3.9	13.8 ± 1.0	55.2 ± 2.9
QAOA	55.0 ± 3.2	19.1 ± 2.6	73.7 ± 7.2

The distinction between Table 5 (median/IQR across varying base sizes) and Table 6 (mean/std at fixed $n = 8$) reveals important compiler scaling dynamics. At $n = 8$, Qiskit’s latency increases moderately on parameterized workloads (55.0 ± 3.2 ms on QAOA) because Qiskit’s transpiler executes routing and variable-binding passes over larger target registers. Cirq exhibits exceptional stability, completing every 8-qubit workload in under 33 ms with tight standard deviations (≤ 2.6 ms), owing to its lightweight moment-insertion model. PennyLane experiences dramatic growth on oracle algorithms (184.2 ± 6.5 ms on Grover’s, 106.3 ± 3.4 ms on Deutsch–Jozsa), driven by eager tape expansion during QNode evaluation when multi-controlled gates are expanded into hundreds of primitive operations. For right-skewed distributions such as Bit-Flip error correction and initial PennyLane evaluation, median and IQR provide a more robust central tendency against OS scheduling and Python first-import warm-up (runs 2–10 consistently execute faster than run 1).

8.3 Structural Benchmarks and Gate Normalization Analysis

To understand the topological transformations induced by each framework’s compilation pipeline, Table 7 summarizes the structural properties (N_{gates} , depth D , and multi-qubit gate ratio R_{mq}) emitted by our adapters at each algorithm’s evaluated register size (Report Table 4.4).

Table 7. Structural benchmark summary across framework adapters at each algorithm’s evaluated register size (Report Table 4.4). Shows Qubits (n), Total Gates (N_{gates}), Circuit Depth (D), and Multi-Qubit Ratio (R_{mq}).

Algorithm	Qiskit Adapter				Cirq Adapter			PennyLane Adapter			
	n	N_{gates}	D	R_{mq}	N_{gates}	D	R_{mq}	N_{gates}	D	R_{mq}	
Bell State	2	4	3	0.25	4	3	0.25	4	3	0.25	
GHZ State	8	6	4	0.33	6	4	0.33	16	9	0.44	
Grover’s Algorithm	6	18	12	0.11	18	12	0.11	216	62	0.00	
Bernstein–Vazirani	8	13	6	0.15	13	6	0.15	31	4	0.16	
VQE	6	5	2	0.20	5	3	0.20	17	7	0.29	

Table 7 shows that Qiskit and Cirq emit near-identical structural footprints—identical for Bell, GHZ, Grover, and Bernstein–Vazirani (e.g., $N_{\text{gates}} = 18$, $D = 12$ for 6-qubit Grover’s Search), differing only by a single level of depth on VQE. Conversely, PennyLane’s adapter (`pennylane_to_qasm.py`) emits substantially expanded circuit structures, reaching **216 gates and depth 62 on 6-qubit Grover’s Search**, compared to 18 gates and depth 12 across Qiskit and Cirq. This divergence is not an artifact of adapter inefficiency but stems directly from PennyLane’s architecture: during QNode tape construction, complex operations (MultiControlledX, Toffoli) are eagerly decomposed into elementary gates before the compilation plane intercepts the AST. Our `ConversionStats` dataclass

makes this structural expansion transparent to developers prior to simulation dispatch.

8.4 Scaling Behavior and Power-Law Depth Analysis

To predict circuit growth across expanding quantum registers, we fit the emitted circuit depth $D(n)$ over the evaluated $n \in [2, 8]$ qubit sweep to the power-law model (Report Section 4.6.5):

$$D(n) = c \cdot n^a \quad (1)$$

where a is the scaling exponent and c a normalization constant. To avoid conflating fit intercepts with growth behavior, Table 8 reports the fitted exponent a per adapter, together with the resulting scaling class, across core workloads on Machine B.

Table 8. Power-law depth scaling exponents ($D = c \cdot n^a$; exponent a shown) across framework adapters, fitted over the evaluated $n \in [2, 8]$ qubit sweep (Report Table 4.5). Qiskit and Cirq preserve high-level IR gates at constant depth ($a = 0.00$).

Algorithm	Qiskit a	Cirq a	PennyLane a	PennyLane Scaling Class
GHZ State	0.00	0.00	1.00	Linear
Deutsch–Jozsa	0.00	0.00	0.91	Sub-linear
Grover’s Algorithm	0.00	0.00	2.21	Super-linear
Bernstein–Vazirani	0.00	0.00	0.87	Sub-linear

The scaling behavior in Table 8 highlights a stark architectural divide (Figures 4.6–4.7). Qiskit and Cirq adapters preserve high-level semantic gates (GHZ, oracle calls, and diffusers represented as single instructions in OpenQASM 3.0), maintaining strictly constant depth ($a = 0.00$) across register expansion prior to hardware-specific unrolling. In contrast, PennyLane exhibits linear depth scaling on GHZ states ($a = 1.00$), sub-linear scaling on oracle circuits ($a = 0.87, 0.91$), and **super-linear scaling on Grover’s Algorithm** ($a = 2.21$). This super-linear exponent confirms that as register width n grows, the eager unrolling of multi-controlled diffusion operators in PennyLane compounds quadratically, placing a heavier computational burden on classical statevector simulation engines.

8.5 Cross-Framework Semantic Equivalence and Quantum Volume Analysis

To confirm that compilation to OpenQASM 3.0 preserves exact quantum state semantics regardless of source framework or structural expansion, we evaluated

Table 9. Pairwise Jensen–Shannon Divergence across 7 representative algorithm configurations at $N = 4,096$ shots on Machine B (Report Table 4.6). Columns give the three framework pairings; all fall below the threshold $\text{JSD} < 0.02$.

Algorithm	n	JSD (Q–C)	JSD (Q–PL)	JSD (C–PL)	All < 0.02 ?
Bell State	2	0.0035	0.0002	0.0037	✓ Pass
GHZ State	3	0.0005	0.0021	0.0017	✓ Pass
QML XOR Classifier	2	0.0018	0.0049	0.0067	✓ Pass
QML XOR Classifier	4	0.0010	0.0017	0.0007	✓ Pass
VQE	2	0.0033	0.0029	0.0053	✓ Pass
QAOA	2	0.0035	0.0035	0.0049	✓ Pass
Grover’s Algorithm	2	0.0000	0.0000	0.0000	✓ Pass

output distribution agreement across our benchmark suite on Machine B (backend="cirq", $N = 4,096$ shots). Table 9 summarizes the Jensen–Shannon Divergence (JSD) [8] across 7 representative core algorithms (Report Table 4.6).

Across every pairing in Table 9, divergence remains extremely low (maximum $\text{JSD} = 0.0067$, far below the analytical threshold $\text{JSD} < 0.02$), confirming pairwise semantic equivalence for these representative algorithms. Across the full 12-algorithm suite, **9 algorithms pass directly** under $\text{JSD} < 0.02$ (the 7 configurations in Table 9, plus Deutsch–Jozsa and Bernstein–Vazirani, which reach $\text{JSD} = 0.000$); the remaining 3 are resolved by the register-convention analysis below.

Analytical JSD threshold and measurement-register analysis. Under multinomial finite sampling ($N = 4,096$ shots across K outcomes), the expected divergence between two draws of an identical distribution is approximately $\mathbb{E}[\text{JSD}] \approx \frac{K-1}{4N \ln 2}$ (≈ 0.000088 for $K = 2$; ≈ 0.00132 for $K = 16$), so $\text{JSD} < 0.02$ conservatively isolates finite-shot noise from true compiler anomalies. For the 3 algorithms in the extended 12-algorithm suite that initially exhibited large raw divergences (Quantum Teleportation $\text{JSD} \approx 1.0$; Bit-Flip and Phase-Flip Codes $\text{JSD} \approx 0.70$), we show that all three stem from structural register conventions rather than semantic compilation errors:

1. **Quantum Teleportation (1-bit vs. 3-bit projection):** PennyLane explicitly measures only Bob’s target wire (`qml.probs(wires=[2])`), yielding a 1-bit support ('0': 2056, '1': 2040). Qiskit and Cirq measure the full 3-qubit register (wires 0, 1, 2). Marginalizing the Qiskit/Cirq 3-bit distribution onto wire 2 yields $\Pr(|0\rangle) = 0.502$, $\Pr(|1\rangle) = 0.498$, matching PennyLane with $\text{JSD} = 0.00012$.
2. **Bit-Flip and Phase-Flip Codes (Endianness permutation):** Qiskit and PennyLane measure individual bits in register order (`c[i] = measure q[i]`), producing 010. Cirq’s adapter emits vector measurements (`cirq.measure(q0, q1, q2)`), ordering bits in reverse physical endianness (101). Applying a deterministic bit reversal ($d_2 d_1 d_0 \rightarrow d_0 d_1 d_2$) immediately reduces divergence from 0.706 to $\text{JSD} = 0.00035$.

Quantum Volume and the structural-not-semantic argument. To provide further evidence that structural differences between adapters (N_{gates} , depth, power-law scaling) do not corrupt the underlying quantum information, we conducted a Quantum Volume (QV) analysis across all three framework adapters (Report Table 4.7), shown in Table 10.

Table 10. Quantum Volume analysis across framework adapters executed via the QSim statevector kernel (Report Table 4.7). Identical QV and Bell-state fidelity across adapters indicate a consistent underlying execution substrate.

Framework Adapter	Effective QV	$\log_2(\text{QV})$	Bell Fidelity
Qiskit Adapter	2048	11	0.9786
Cirq Adapter	2048	11	0.9786
PennyLane Adapter	2048	11	0.9786

The exact match across all three framework adapters in effective Quantum Volume ($\text{QV} = 2048$, $\log_2(\text{QV}) = 11$) and Bell-state fidelity (0.9786) under uniform QSim execution supports our core thesis: **structural divergences across frameworks stem from high-level transpilation and decomposition strategies rather than numerical simulator discrepancies or loss of quantum-state fidelity.**

8.6 Holistic Framework Scoring (QPack Benchmark)

To provide a unified metric comparing adapter efficiency, we evaluated the frameworks using our holistic QPack scoring formulation across compilation speed (S_{compile}), depth scaling (S_{scale}), accuracy (S_{accuracy}), and capacity (S_{capacity}). Table 11 details the breakdown (Report Table 4.8).

Table 11. QPack holistic benchmark scores across Qiskit, Cirq, and PennyLane (Report Table 4.8). Component sub-scores (S_{compile} , S_{scale} , S_{accuracy} , S_{capacity}) combine into a weighted overall score (Q_{pack} ; higher is better).

Framework	S_{compile}	S_{scale}	S_{accuracy}	S_{capacity}	Overall (Q_{pack})
Qiskit	2.84	3.91	9.41	8	58.80
Cirq	2.84	3.91	9.41	8	58.79
PennyLane	2.56	-0.90	9.18	2	9.28

Qiskit (**58.80**) and Cirq (**58.79**) achieve virtual parity—their sub-scores are identical and the residual 0.01 gap reflects rounding in the weighted composite—demonstrating balanced strength across compilation throughput, constant IR

depth scaling, accuracy, and capacity. PennyLane’s overall score (**9.28**) is dominated by its negative scaling penalty ($S_{\text{scale}} = -0.90$, from super-linear Grover depth growth) and reduced capacity ($S_{\text{capacity}} = 2$), though its differentiable execution remains valuable for variational and QML workloads.

8.7 Simulation Engine Orchestration Performance and Throughput (QSim Engine)

We evaluated QSim’s back-end execution performance and throughput on Machine B (Ryzen 7 7700, 32 GB RAM) (Report Section 4.6.8 / NFR-P2 / NFR-P3):

- **High-Throughput Simulation (NFR-P2):** A 6-qubit Grover’s Search circuit across 4,096 shots completed execution in exactly **782 ms** (consistently under 800 ms), outperforming our system requirement of < 5.0 seconds by more than $6\times$.
- **Visualization Rendering (NFR-P3):** Real-time D3.js/SVG processing and DOM construction for circuits up to 50 gates completed in **110 ms** (well within our sub-200 ms interactivity threshold).
- **Round-Trip IR Accuracy:** End-to-end QASM parsing and reconstruction across all standard gate operations maintained $\text{JSD} < 0.007$.

Table 12 reports QSim’s end-to-end wall-clock scaling from $n = 4$ to $n = 20$ qubits executing GHZ states via statevector (backend="cirq", 4,096 shots).

Table 12. End-to-end QSim simulation wall-clock time vs. qubit count (n) for statevector simulation of GHZ states across 4,096 shots (backend="cirq"). Median and IQR over 10 runs.

Qubit Count (n)	4	8	12	16	18	20
Median Time (ms)	31.8	57.1	84.0	108.8	127.4	162.4
IQR (ms)	0.4	1.8	1.6	1.6	1.5	1.6

Orchestration latency scales smoothly from 31.8 ms at $n = 4$ to 162.4 ms at $n = 20$ ($2^{20} = 1,048,576$ complex amplitudes), where AST traversal and visitor reconstruction overhead (~ 15 – 20 ms) transition to statevector memory complexity.

8.8 Compilation-to-Execution Baseline Analysis

To benchmark QCanvas’s IR-first decoupling against native framework execution, we compared native Cirq execution against QCanvas single-call orchestration on a 2-qubit Bell State ($N = 4,096$ shots, Machine A). Table 13 details the breakdown.

Table 13. Baseline comparison: native framework invocation vs. QCanvas single-call compile-and-execute orchestration (Bell State, 2q, 4,096 shots, Machine A).

Execution Workflow	Median Time (ms)	Share of Single-Call
Native Cirq Circuit Build + Simulator.run()	6.5	—
QCanvas compile_and_execute() (Single-Call)	31.8	+25.3 ms vs. native
– Adapter AST $\rightarrow$ OpenQASM 3.0 String	≈ 8.1	(25.5%)
– IR Validation & PyQASM AST Traversal	≈ 10.2	(32.1%)
– Visitor Reconstruction & run_qasm	≈ 13.5	(42.5%)

The QCanvas single-call time (31.8 ms) decomposes across string serialization (8.1 ms, 25.5%), validation and `pyqasm` parsing (10.2 ms, 32.1%), and visitor reconstruction plus execution (13.5 ms, 42.5%). The resulting ~ 25.3 ms overhead over native Cirq is an immaterial trade-off for multi-framework interoperability and inspectable execution traces.

8.9 Testing, Verification, and Correctness Assurance

To guarantee platform stability across distributed services, QCanvas maintains an extensive, multi-tier testing regimen. Table 14 summarizes our automated test suite and verification matrix (Report Table 4.2).

Table 14. Summary of QCanvas automated validation suite and verification matrix across compilation, validation, and orchestration layers (Report Table 4.2). Pass rate: 100%.

Test Category / Phase	Passing Tests	Coverage Focus & Verification Scope
Iteration 1 Unit Tests	73	Fundamental gate conversions, AST parsing, token checks.
Iteration 2 Unit Tests	151	Advanced OpenQASM 3.0 features, control flow, subroutines, edge cases.
System & Integration Suite	80	End-to-end API, database persistence, Web-Socket, Redis caching.
Automated Test Suite Total	304	100% automated suite pass rate across all services.
Synthetic Stress Matrix	2,160	Multi-qubit ($2 \leq n \leq 15$) and multi-shot ($1 \leq N \leq 10^4$) sweeps across all 3 adapters.
Compact IR Representations	50	Gate vocabulary normalization and equivalence verification.
Official Demonstration Suite	16	End-to-end platform validation across canonical algorithms.
Grand Total Test Matrix	2,530	100% verified execution across all test configurations.

Furthermore, QSim’s visitor and simulation engine layer is backed by over **1,400+ total unit test cases** (QSim Report Table 4.3), including **450+ dedi-**

cated unit tests verifying each of CirqVisitor, QiskitVisitor, and PennyLaneVisitor, supporting robust AST traversal and instruction mapping.

8.10 Non-Functional Requirements (NFRs) Empirical Validation

To verify operational readiness across deployment dimensions, Table 15 presents the empirical validation summary of our Non-Functional Requirements (Report Table 4.9), measured on Machine B.

Table 15. Empirical validation summary of Non-Functional Requirements (Report Table 4.9). Replaces target thresholds with observed system measurements across all criteria.

NFR Identifier & Category	Target Threshold	Observed Empirical Measurement	Mea- Status
NFR-R1 (Reliability)	99.9% system uptime across container services.	99.94% measured uptime across Backend, PostgreSQL, and Redis containers under stress load.	✓ Pass
NFR-U3 (Usability)	Hover documentation response time < 200 ms.	Documentation returned within 45 ms for contextual gate unitary matrices and Bloch explanations.	✓ Pass
NFR-P1 (Conversion Latency)	< 1,500 ms for circuits under 100 gates.	Cirq: 8–13 ms; Qiskit: 7–55 ms; PennyLane: 12–184 ms across all standard workloads.	✓ Pass
NFR-P2 (Simulation Throughput)	End-to-end execution < 5.0 seconds.	6-qubit Grover’s Search (4,096 shots) completed in exactly 782 ms on QSim statevector.	✓ Pass
NFR-P3 (Visualization Render)	SVG rendering < 200 ms for < 50 gates.	Render processing completed within 110 ms across D3.js DOM generation.	✓ Pass
NFR-S1 (Security & Rate Limiting)	Enforce 20 requests/min per client IP.	HTTP 429 Too Many Requests triggered immediately upon exceeding 20 req/min.	✓ Pass

8.11 Comparison with Related Tools

QCanvas does not aim to supersede specialized optimizing compilers like t|ket> in minimizing gate depth. Rather, its contribution lies in eliminating the operational friction between framework compilation, IR normalization, and multi-backend execution inside an inspectable workbench. Where graphical editors like IBM Quantum Composer [16] or Quirk [14] are either vendor-locked or lack SDK integration, and where command-line transpilers require manual pipeline glue,

QCanvas provides a single, verifiable compile-and-execute contract across the NISQ ecosystem.

9 Discussion

9.1 Architecture Trade-offs: IR-First vs. Direct Framework Routing

Choosing OpenQASM 3.0 as the intermediate representation provides full expressiveness—including classical control flow, gate modifiers, and complex data types—and establishes an inspectable, human-readable contract between compilation and execution. The trade-off is threefold: (1) a round-trip through the QASM text format introduces a serialization step (~ 8.1 ms baseline overhead) that direct memory-to-memory framework routing would avoid; (2) while our latest `RunArgs` API decouples target simulation representation methods (`statevector`, `density_matrix`, `mps`, `chp`) from host sub-engines (Table 2), individual sub-engines still require explicit capability validation before executing non-default mathematical kernels; and (3) OpenQASM 3.0 does not cover pulse-level control (OpenPulse serves that role), precluding microwave pulse-shaping compilation in the current design.

9.2 PennyLane Gate Expansion and Its Implications

PennyLane’s `QNode` tape fully decomposes multi-controlled gates during tape construction before the AST is passed to the compilation plane. For Grover’s Search at 6 qubits, this yields **216 emitted gates** ($D = 62$) **versus 18 gates** ($D = 12$) **for Qiskit and Cirq**. This is a deliberate architectural choice within PennyLane (full decomposition ensuring cross-hardware execution portability) rather than a QCanvas inefficiency. However, it means the OpenQASM 3.0 emitted from the PennyLane adapter contains substantially larger circuit structures for oracle and multi-controlled workloads. The `ConversionStats` data model makes this structural expansion transparent: users and automated schedulers receive precise gate counts, depths, and multi-qubit ratios before submitting simulations.

9.3 Limitations

- **Classical simulation focus.** While QCanvas generates standardized OpenQASM 3.0 suitable for physical hardware execution, QSim currently dispatches exclusively to classical simulation sub-engines and cloud worker clusters rather than physical quantum processing units (QPUs).
- **Equivalence check reference engine.** Our statistical semantic equivalence evaluation (Section 8.5) executes compiled IR strings across a single reference sub-engine (`backend="cirq"`) to isolate compilation differences from simulator backend discrepancies. Truly independent full-stack validation requires cross-sub-engine matrix execution across all target noise models.

- **Cirq-RAG assistant dependencies.** Users without active AWS credentials or Amazon Bedrock access cannot invoke the optional AI code explanation service (`qcanvas_cirq_agent`), though all core IDE and simulation workflows operate independently.

10 Conclusion

QCanvas provides a four-layer, end-to-end quantum software engineering platform that bridges the compile–execute gap across the fragmented NISQ ecosystem by unifying multi-framework AST compilation, OpenQASM 3.0 normalization, and distributed simulation orchestration into a single inspectable architecture. The compilation plane translates Qiskit, Cirq, and PennyLane circuits into validated OpenQASM 3.0 via three specialized AST visitors. QSim’s plugable sub-engine architecture routes the resulting IR to Cirq, Qiskit Aer, or PennyLane simulation kernels across four mathematical representation methods (`statevector`, `density_matrix`, `mps`, and `chp`) through an asynchronous RabbitMQ and Kubernetes pipeline designed to sustain over **100 concurrent jobs** (requirement PERF-3).

Empirical evaluations across **136 distinct parameter configurations** and **2,530 verified test scenarios (100% pass rate)** confirm platform performance, accuracy, and reliability. Measured compilation latencies complete in **8–13 ms median for Cirq, 7–55 ms for Qiskit, and 12–184 ms for PennyLane** across standard 8-qubit workloads. End-to-end simulation throughput achieves **782 ms on 6-qubit Grover’s Search** (< 800 ms), while real-time SVG circuit visualization renders within **110 ms**. Cross-framework semantic equivalence holds across all **12 canonical benchmark workloads**: 9 algorithms satisfy $\text{JSD} < 0.02$ directly (maximum 0.0067), while the remaining 3 reduce to $\text{JSD} \leq 0.00035$ once measurement-register projection and bit-index reversal are applied, reinforced by exact parity across all framework adapters in effective Quantum Volume ($\text{QV} = 2048$) and Bell state fidelity (0.9786). Distributed via two independent PyPI packages (`qcanvas-sdk` and `fastqsim`) alongside an open Next.js Web IDE with **40+ pre-built quantum algorithm templates across 14 application categories**, QCanvas serves as a practical, empirically validated tool for quantum software developers, researchers, and educators.

Future releases will extend QCanvas to physical QPU dispatch via cloud provider APIs, expand the simulation plane with GPU-accelerated tensor-network kernels, and introduce native compilation adapters for Amazon Braket and Stim.

References

1. Preskill, J.: Quantum computing in the NISQ era and beyond. *Quantum* **2**, 79 (2018). <https://doi.org/10.22331/q-2018-08-06-79>

2. Cross, A., et al.: OpenQASM 3: A broader and deeper quantum assembly language. *ACM Trans. Quantum Comput.* **3**(3), 12 (2022). <https://doi.org/10.1145/3505636>
3. Qiskit contributors: Qiskit: An open-source framework for quantum computing. Zenodo (2023). <https://doi.org/10.5281/zenodo.2573505>
4. Cirq Developers: Cirq. Zenodo (2024). <https://doi.org/10.5281/zenodo.8161858>
5. Bergholm, V., et al.: PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv:1811.04968* (2022)
6. Sivarajah, S., et al.: t|ket): A retargetable compiler for NISQ devices. *Quantum Sci. Technol.* **6**(1), 014003 (2020). <https://doi.org/10.1088/2058-9565/ab8e92>
7. Gamma, E., Helm, R., Johnson, R., Vlissides, J.: *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley (1994)
8. Lin, J.: Divergence measures based on the Shannon entropy. *IEEE Trans. Inf. Theory* **37**(1), 145–151 (1991). <https://doi.org/10.1109/18.61115>
9. Nielsen, M.A., Chuang, I.L.: *Quantum Computation and Quantum Information*, 10th edn. Cambridge University Press (2010)
10. Aaronson, S., Gottesman, D.: Improved simulation of stabilizer circuits. *Phys. Rev. A* **70**(5), 052328 (2004). <https://doi.org/10.1103/PhysRevA.70.052328>
11. Burgholzer, L., Wille, R.: Advanced equivalence checking for quantum circuits. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **40**(9), 1810–1824 (2021). <https://doi.org/10.1109/TCAD.2020.3032630>
12. Younis, E., et al.: BQSKit: Extensible Quantum Circuit Synthesis. In: *Proc. IEEE/ACM SC22 Workshops*. IEEE (2022). <https://doi.org/10.1109/SCW55976.2022.00012>
13. Amy, M., et al.: staq—A full-stack quantum processing toolkit. *Quantum Sci. Technol.* **5**(3), 034016 (2020). <https://doi.org/10.1088/2058-9565/ab9359>
14. Gidney, C.: Quirk: A drag-and-drop quantum circuit simulator. <https://algassert.com/quirk> (2017)
15. pyqasm contributors: pyqasm—Python utilities for OpenQASM 3.0. <https://github.com/qBraid/pyqasm>
16. IBM Quantum: IBM Quantum Composer. <https://quantum.ibm.com/composer> (2024)